

TW1.3 - Docker & Jenkins Freestyle Project Report

Name: JYOTI KUMARI SAH

PRN: 23070122278

Clas:CS B

Subject: DevOps Lab

Aim

To build and deploy an application using Docker and Jenkins Freestyle Pipeline.

Objectives

Understand containerization, create Docker images, configure Jenkins jobs and automate builds.

Theory

Docker is a containerization platform that packages applications with their dependencies into lightweight containers. Jenkins is an automation server used for Continuous Integration and Continuous Deployment (CI/CD). Combining Docker with Jenkins enables automated building, testing and deployment of applications.

Software Requirements

Docker Desktop, Jenkins, Git, Java, Maven, GitHub.

Docker Concepts

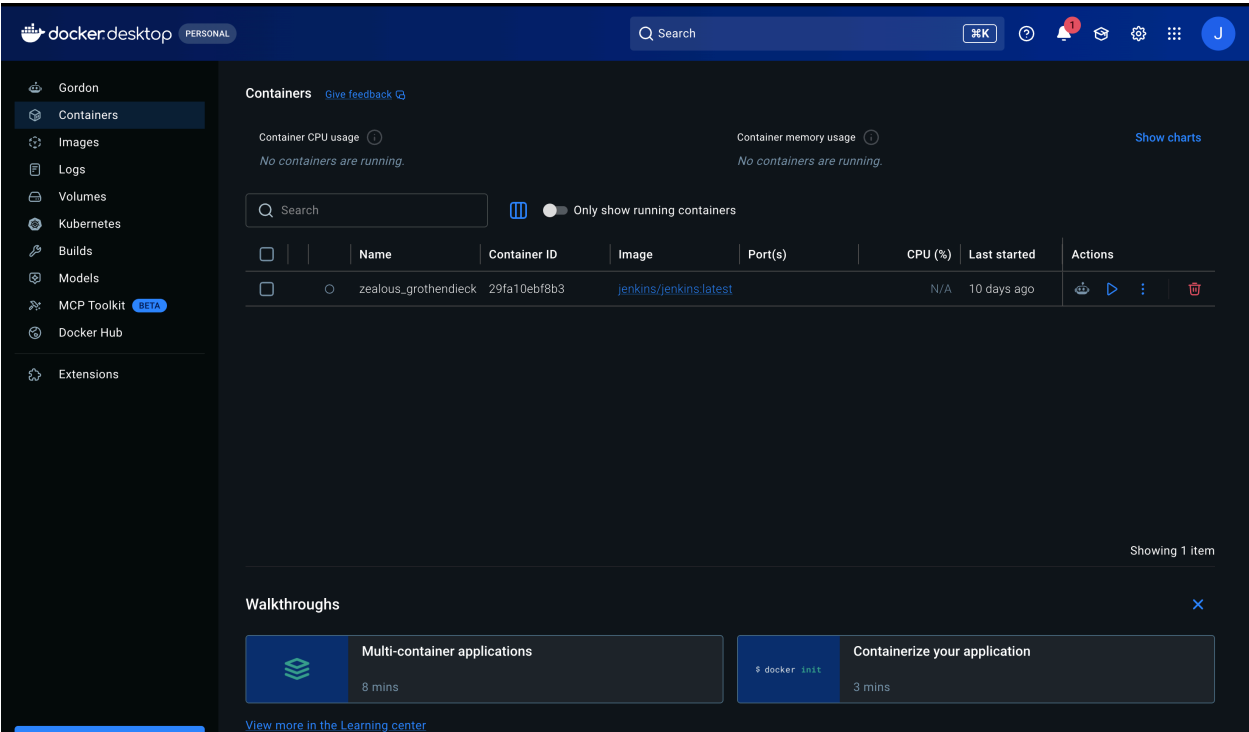
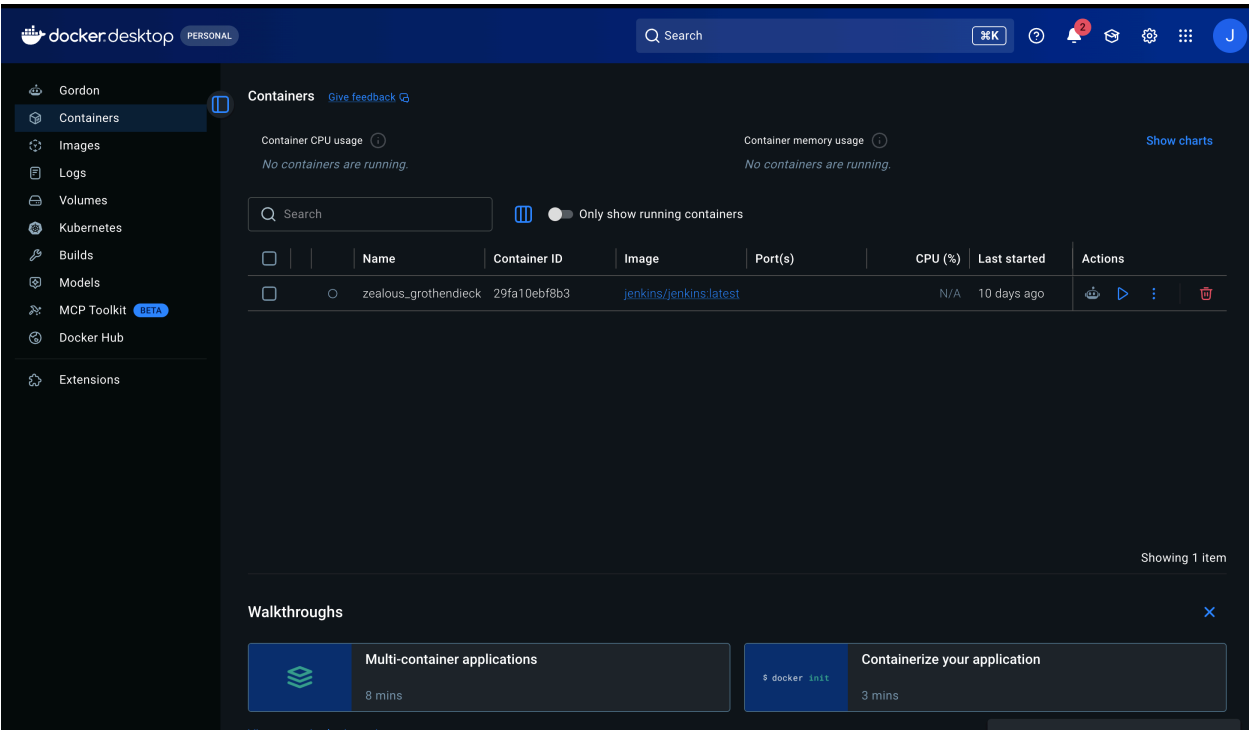
Explain Docker Image, Container, Dockerfile, Docker Hub, Volumes, Networks and Bind Mounts.

Jenkins Concepts

Explain Freestyle Jobs, Build Triggers, Workspaces, Plugins, Build Steps and Post-build Actions.

Procedure

Install Docker and Jenkins, create Dockerfile, build image, configure Jenkins job, connect GitHub repository, execute build, verify successful deployment.



Jenkins

All

New Item

🔍

⚙️

👤

New Item

Enter an item name

my-CICD-pipeline

Select an item type

Pipeline

Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.

Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

Multi-configuration project

Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

Folder

Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

Multibranch Pipeline

Creates a set of Pipeline projects according to detected branches in one SCM repository.

Organization Folder

OK

Jenkins

+ New Item

📄 Build History

Build Queue

No builds in the queue.

Build Executor Status

0/2

All

+

S	W	Name ↓	Last Success	Last Failure	Last Duration	
☁️	☀️	my-CICD-pipeline	N/A	N/A	N/A	▶️

Icon:

S

M

L

⋮

🔍

⚙️

👤

📝 Add description

The screenshot shows the Jenkins web interface. At the top, the Jenkins logo is on the left, and search, settings, and user icons are on the right. Below the logo, the breadcrumb 'my-CICD-pipeline' is followed by a dropdown arrow, and '#1' is also followed by a dropdown arrow. On the left sidebar, there are links for 'Status', 'Changes', 'Console Output' (which is highlighted), 'Delete build '#1'', and 'Edit build information'. The main area is titled 'Console' and contains a terminal window with the following output:

```
Started by user Jyoti Sah
Running as SYSTEM
Building in workspace /Users/jyotisah/.jenkins/workspace/my-CICD-pipeline
[my-CICD-pipeline] $ /bin/sh -xe /var/folders/r8/7h8ypt314gqc4nczrbm241140000gn/T/jenkins9220183915508167327.sh
+ echo 'Hello DevOps Lab'
Hello DevOps Lab
+ pwd
/Users/jyotisah/.jenkins/workspace/my-CICD-pipeline
+ ls -la
total 0
drwxr-xr-x  2 jyotisah  staff   64 Aug  3 14:23 .
drwxr-xr-x  3 jyotisah  staff  96 Aug  3 14:23 ..
Finished: SUCCESS
```

At the top right of the console output area, there are three buttons: 'Download', 'Copy', and 'View as plain text'.

Commands Used

docker build, docker run, docker ps, docker images, docker stop, docker rm, docker logs, git clone, mvn clean package.

Learning Outcomes

Students learned Docker fundamentals, Jenkins automation and CI/CD workflow.

Conclusion

Docker and Jenkins together automate software delivery, improve consistency and reduce manual effort.